

Control Statements in C

In C programming, a control statement is a statement that alters the flow of execution of a program. We use control statements to determine the order in which the instructions within a program are executed. They allow us to make decisions, repeat actions, and control the flow of our program based on certain conditions.

Types of Control Statements in C

- **Selection Statements (Conditional Statements)**
- **Iteration Statements (Loops)**
- **Jump Statements (Un-Conditional Statements)**

1. Selection Statements (Conditional Statements)

Selection or conditional statements are a fundamental part of C programming language. They allow you to make decisions in your code by executing certain blocks of code based on the evaluation of conditions. We have primarily four types of selection statements: **if**, **if-else**, **if-else-if** & **switch**.

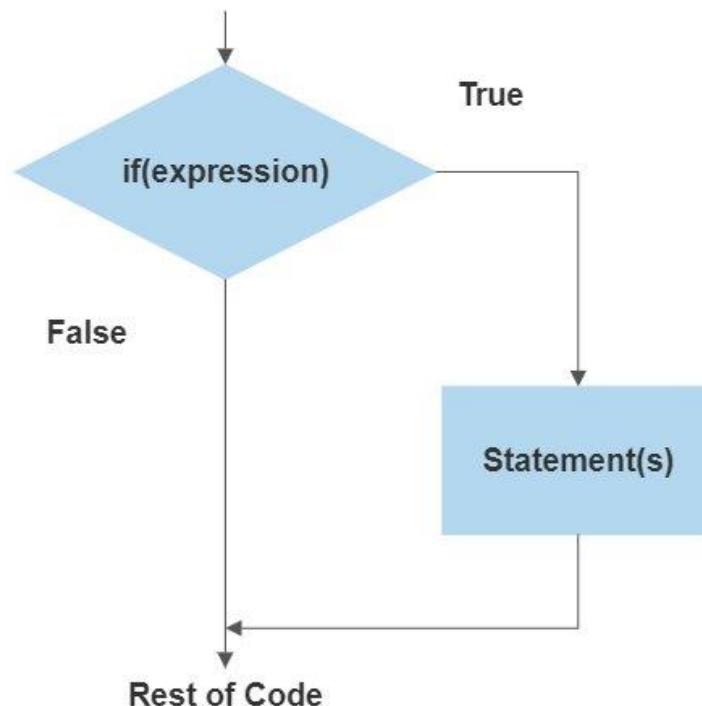
A. if statement in C

In the C programming language, we use the **if** statement for conditional branching. It allows a program to evaluate if a given condition is true or false and execute a block of code accordingly.

Syntax

```
if (condition)
{
    // statements to execute if condition is true
}
```

Flowchart



For example

```
#include <stdio.h>

int main() {
    if (25 > 17) {
        printf("25 is greater than 17");
    }
    return 0;
}
```

Output

25 is greater than 17

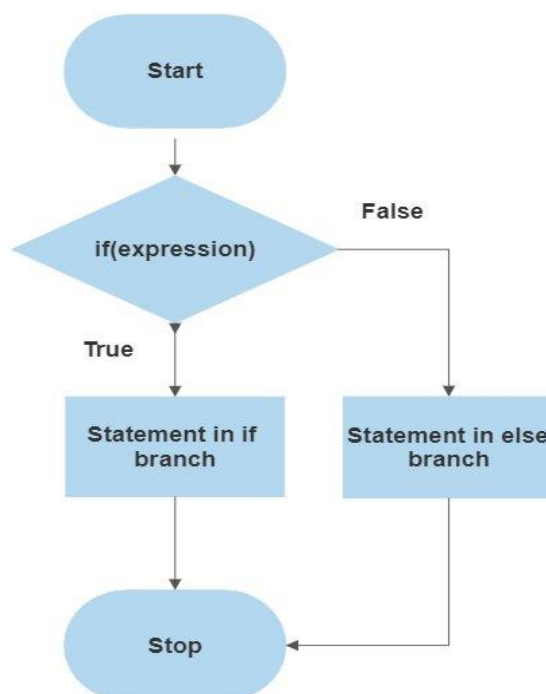
B. if-else statement in C

The if-else statement in C is a fundamental control flow structure that allows for the conditional execution of code. It tests a condition: if the condition is true, one block of code is executed, and if the condition is false, another block (or none at all) is executed.

Syntax

```
if (condition)
{
    // statements to execute if condition is true
}
else
{
    // statements to execute if condition is false
}
```

Flowchart



For example, let's understand this with the help of a scenario

Imagine you are using a ride-sharing app like pathao or in-drive. Sometimes, when the demand for rides is high, these apps activate something called “surge pricing”. This means that the cost of a ride might be higher than usual to balance the high demand with the available drivers.

```
#include <stdio.h>

int main() {
    int surgePricingActive = 1;

    if (surgePricingActive) {
        printf("Surge pricing is active. Fares might be higher now.");
    } else {
        printf("No surge pricing. Good time to book a ride!");
    }

    return 0;
}
```

Output

Surge pricing is active. Fares might be higher now.

Nested if-else-if statement in C

In programming, if-else-if statement is also known as if-else-if ladder. It is used when there are more than two possible action based on different conditions.

Syntax

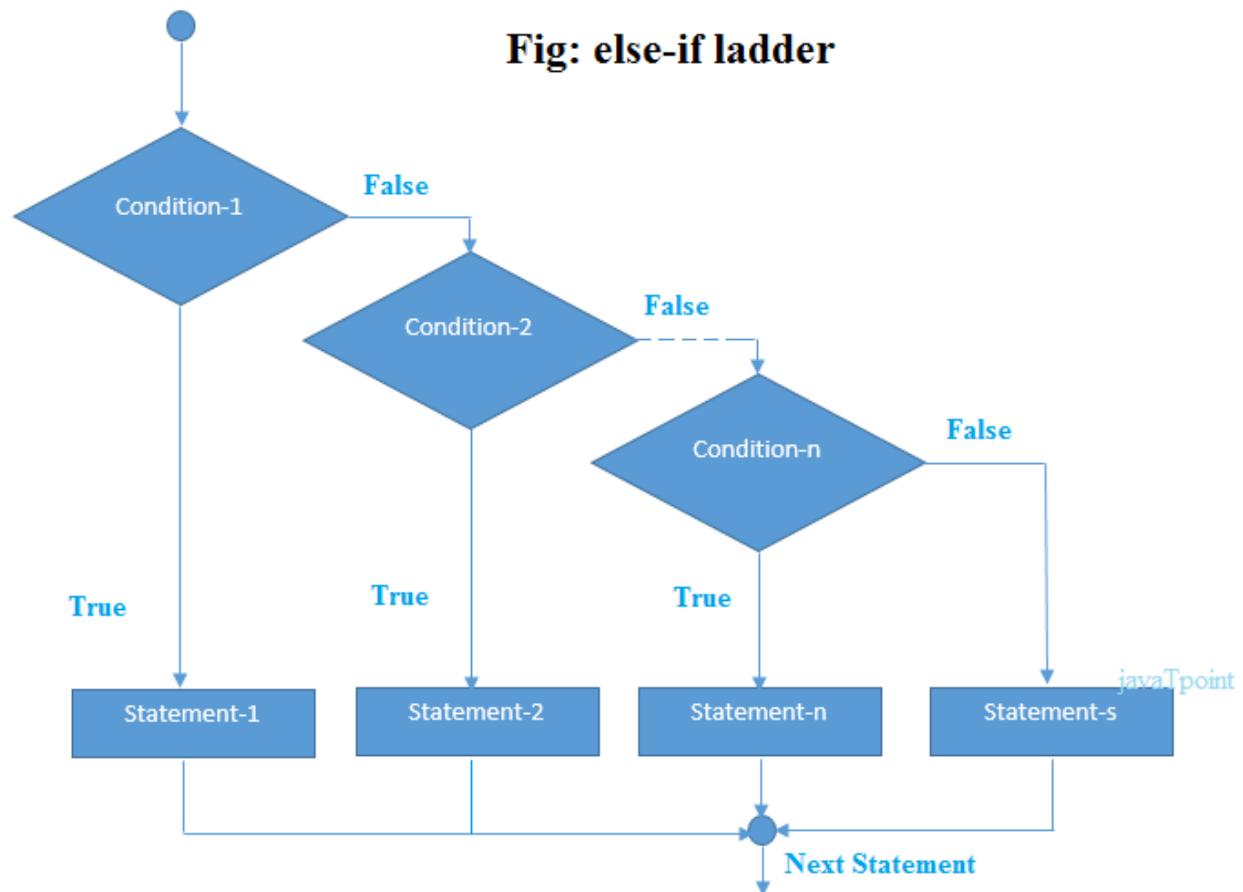
```
if (Condition1)
{
    Statements; // statements to execute if condition1 is true
}
else if(Condition2)
{
    Statement2; // statements to execute if condition2 is true
}
.
.
.
else if(ConditionN)
{
```

```

    StatementN; // statements to execute if conditionN is true
}
else
{
    Default_Statement; // statements to execute if all the above conditions are false
}

```

Flowchart



Example: C program to print weekday based on given number.

```

#include <stdio.h>

int main()
{
    int day;
    printf("Enter day number: ");
    scanf("%d", &day);
    if(day==1)
    {
        printf("SUNDAY.");
    }
}

```

```
}  
    else if(day==2)  
    {  
        printf("MONDAY.");  
    }  
    else if(day==3)  
    {  
        printf("TUESDAY.");  
    }  
    else if(day==4)  
    {  
        printf("WEDNESDAY.");  
    }  
    else if(day==5)  
    {  
        printf("THURSDAY.");  
    }  
    else if(day==6)  
    {  
        printf("FRIDAY.");  
    }  
    else if(day==7)  
    {  
        printf("SATURDAY.");  
    }  
    else  
    {  
        printf("INVALID DAY.");  
    }  
    return(0);  
}
```

Output:

Run 1:

Enter day number: 4

WEDNESDAY.

Run 2:

Enter day number: 2

MONDAY.

Example 2: C program to find largest number from four given numbers.

```
#include<stdio.h>
int main()
{
    float a,b,c,d, lg;
    printf("Enter four numbers:\n");
    scanf("%f%f%f%f", &a, &b, &c, &d);

    if(a>b && a>c && a>d)
    {
        lg = a;
    }
    else if(b>a && b>c && b>d)
    {
        lg = b;
    }
    else if(c>a && c>b && c>d)
    {
        lg = c;
    }
    else
    {
        lg = d;
    }
    printf("Largest = %f", lg);
    return(0);
}
```

Output

Enter four numbers:

21.4↵

54.4↵

34.5↵

-67.5↵

Largest = 54.400000

Note: ↵ indicates enter is pressed.

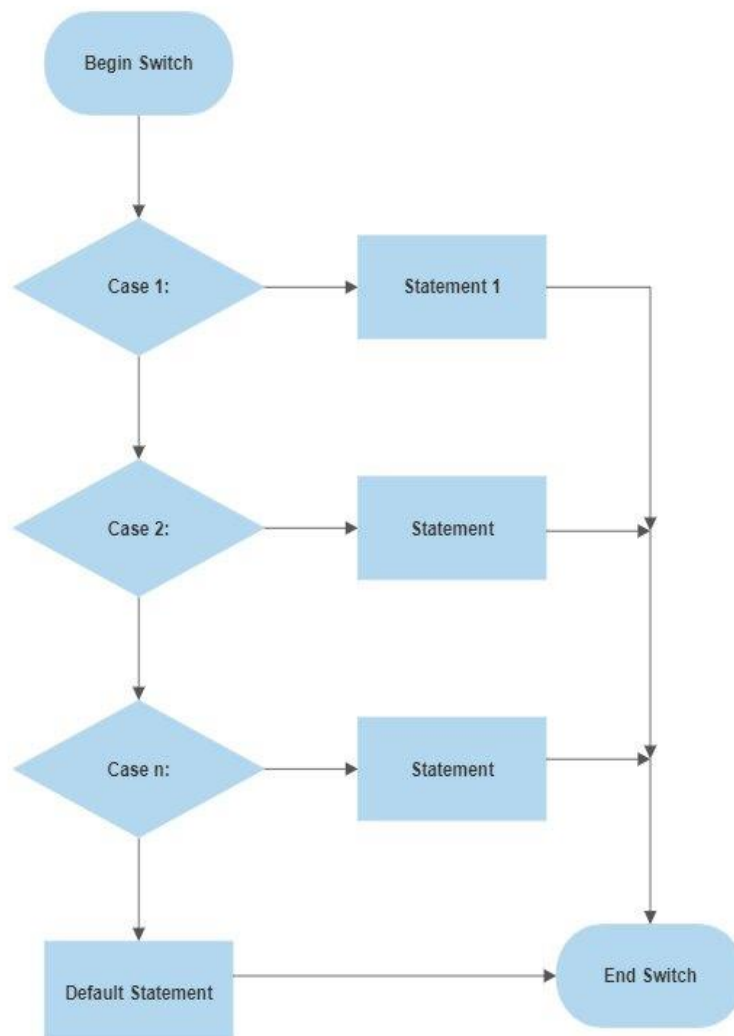
Switch statement in C

The switch statement is a control flow statement in C (and many other programming languages) that allows you to choose one of several possible code blocks to execute based on the value of an expression. It's often used as a more concise alternative to a series of if-else statements when you need to compare a single value against multiple possible values.

Syntax

```
switch (expression) {  
    case value1:  
        // code will be executed if expression equals value1  
        break;  
    case value2:  
        // code will be executed if expression equals value2  
        break;  
    // additional cases...  
    default:  
        // code will be executed if none of the above cases match  
}
```

Flowchart



Let's understand this with the help of a scenario

You have to create a program for a ride-sharing app. The user needs to select a ride type from available options (UberX, UberPool, and UberLux), and the program will provide information about the selected ride type.

```
#include <stdio.h>

int main() {
    char rideType = 'P'; // U for UberX, P for UberPool, L for UberLux

    switch (rideType) {
        case 'U':
            printf("You've selected UberX. A comfortable ride for up to 4.");
            break;
        case 'P':
            printf("You've selected UberPool. Share your ride and save!");
            break;
        case 'L':
            printf("You've selected UberLux. Ride in style!");
            break;
        default:
            printf("Invalid selection. Please choose a valid ride type.");
    }

    return 0;
}
```

Output

You've selected UberPool. Share your ride and save!

In our code, we have a switch statement that uses the value of the rideType variable to determine which ride type the user has selected. Each case corresponds to a different ride type, and the associated code within each case block provides information about the selected ride type. The default case is used to handle an invalid selection by the user.

2. Iteration Statements (Loops)

We use loops to execute a block of code repeatedly based on a condition or a set of conditions. We have primarily three types of iteration statements: **for loop**, **while loop**, **do-while loop**

For loop in C

The **for loop in C** provides a concise way to iterate over a block of code using an initializer, a condition, and an iterator. We commonly use it when the number of iterations is known beforehand.

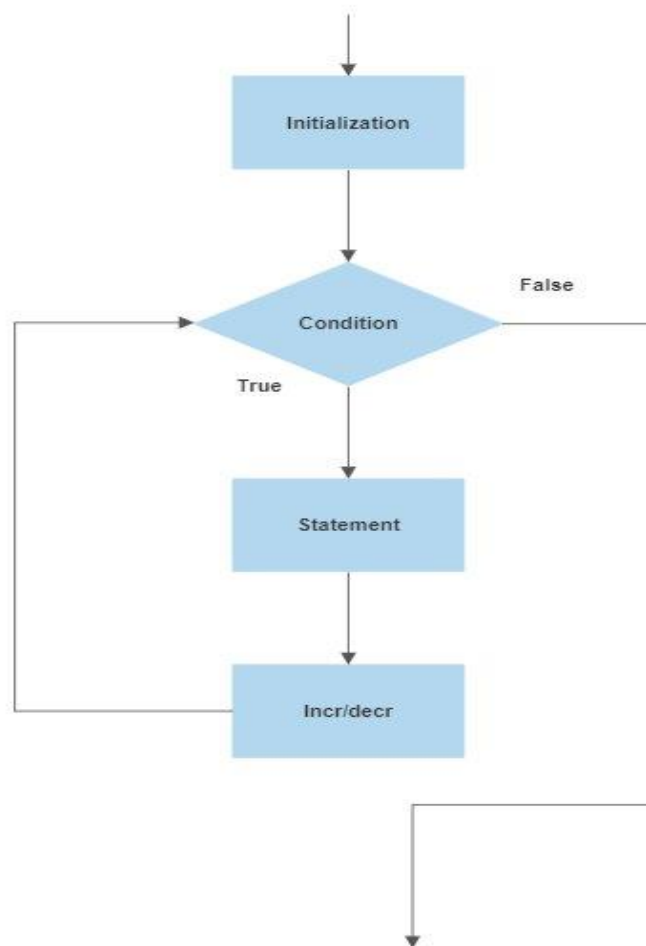
It has three main parts:

- **Initialization:** Executed once at the beginning of the loop. This step allows you to declare and initialize any loop control variables.
- **Condition:** Evaluated before each iteration. The loop body is executed if the condition evaluates to true (non-zero). If it evaluates to false (zero), the loop is terminated.
- **Update:** Executed after each iteration. It's typically used to update loop control variables.

Syntax

```
for (initialization; condition; update)
{
    // code to be executed in each iteration of the loop;
}
```

Flowchart



For example :

```
#include <stdio.h>

int main() {
    for(int i = 1; i <= 7; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Copy code

Output

```
1
2
3
4
5
6
7
```

This output corresponds to the loop iterating through the numbers from 1 to 7 and printing each number on a new line.

while loop in C

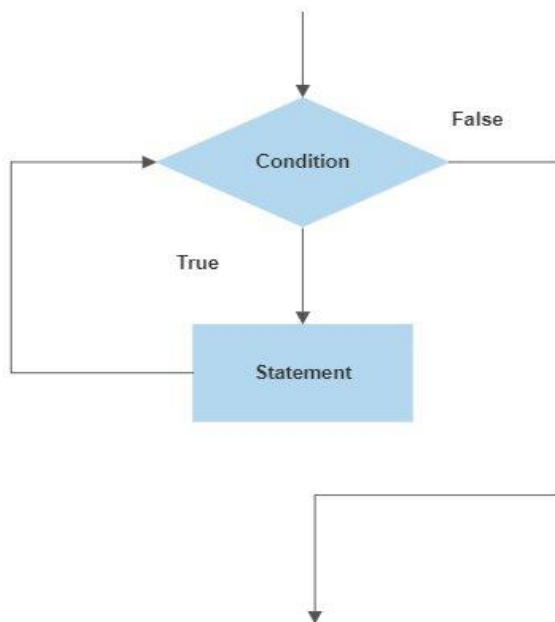
A while loop in C is a control flow structure that allows for a piece of code to be executed repeatedly as long as a particular condition is true.

Here, the **condition** is evaluated before the execution of the loop's body. If the condition is true, the code inside the loop will run. After the loop body has run, the condition is evaluated again, and if it's still true, the body runs again. This process continues until the condition becomes false. If the condition starts off as false, the loop body will never execute.

Syntax

```
while (condition)
{
    // code to be executed as long as condition is true;
}
```

Flowchart



For example, let's understand with the help of a scenario

For example, there's a scenario where you're tracking the distance of an Uber/Ola driver as they approach your location and notify you when the driver has arrived. So, in the below program you know the initial distance of the driver from your location, and the program simulates the driver's progress by decreasing the distance in increments of 200 meters.

```
#include <stdio.h>
int main() {
    int distanceInMeters = 1000;

    while (distanceInMeters > 0) {
        printf("Driver is %d meters away...", distanceInMeters);
        distanceInMeters -= 200; // Assuming the driver covers 200 meters every iteration.
    }
    printf("Your Uber has arrived!");
    return 0;
}
```

Output

Driver is 1000 meters away...
Driver is 800 meters away...
Driver is 600 meters away...
Driver is 400 meters away...
Driver is 200 meters away...
Your Uber has arrived!

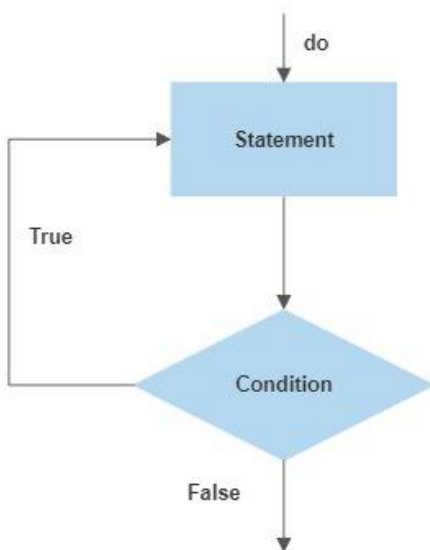
Do-while loop

The do-while loop checks the condition after executing the loop body, which ensures that the loop body is executed at least once, even if the condition is initially false.

Syntax

```
do
{
    // code to be executed;
} while (condition);
```

Flowchart



For example, let's see a scenario-based question

We have to print the multiplication table of N.

```
// C Program to print multiplication table using do-while
// loop
#include <stdio.h>

int main()
{
    int N = 7, i = 1;

    do {
        printf("%d x      %d      =      %d\n", N, i, N * i);
    } while (i++ < 10);

    return 0;
}
```

Copy code

Output

7	x	1	=	7
7	x	2	=	14
7	x	3	=	21
7	x	4	=	28
7	x	5	=	35
7	x	6	=	42
7	x	7	=	49
7	x	8	=	56
7	x	9	=	63
7	x	10	=	70

We printed the table of 7 here using the do-while loop in C.

3. Jump Statements

Jump statements in C provide a way to alter the flow of a program. They “jump” to another part of the code based on certain conditions or instructions. We have primarily four types of jump statements: **break**, **continue**, **goto** & **return**.

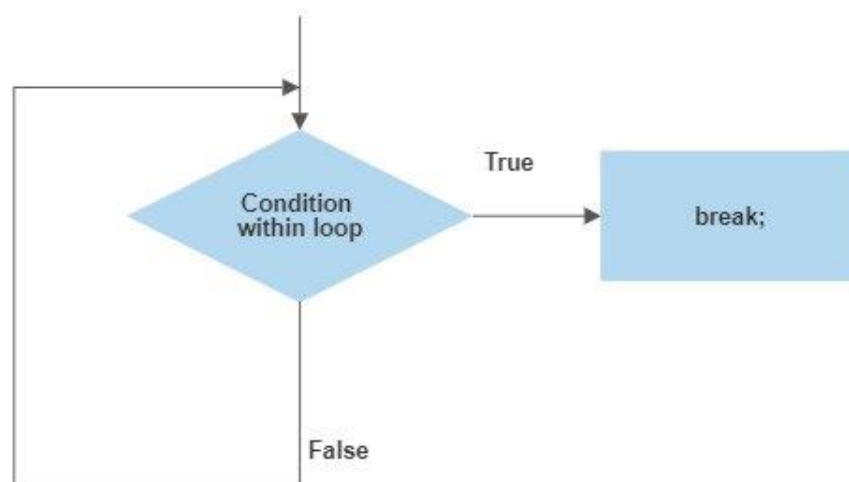
A. break in C

In C, the break statement is used to exit a loop (**for**, **while** or **do-while** loop) or a **switch** statement immediately, without waiting for the loop or switch to finish its normal execution cycle.

Syntax

break;

Flowchart:



For example, let's understand with the help of a scenario

There is a young boy who loves playing games. One day, he decides to try out a Number Guessing Game that he saw in a programming book. So, let's see how we can do that.

```

#include <stdio.h>

int main() {
    int targetNumber= 4;
    int guess= 4;

    printf("Welcome to the Number Guessing Game!
");

    while (1) { // Infinite loop until the correct number is guessed
        printf("Enter your guess: ");
        scanf("%d", &guess);

        if (guess == targetNumber) {
            printf("Congratulations! You guessed the correct number.
");
            break; // Exit the loop since the correct number is guessed
        } else if (guess < targetNumber) {
            printf("Try a higher number.
");
        } else {
            printf("Try a lower number.
");
        }
    }

    printf("Thank you for playing!
");

    return 0;
}

```

Output

Welcome to the Number Guessing Game!
Enter your guess: Congratulations! You guessed the correct number.
Thank you for playing!

The code uses the break statement to terminate the loop when the player guesses the correct number. The game will continue prompting the player for guesses until they guess the target number, making it a functional and interactive number-guessing game.

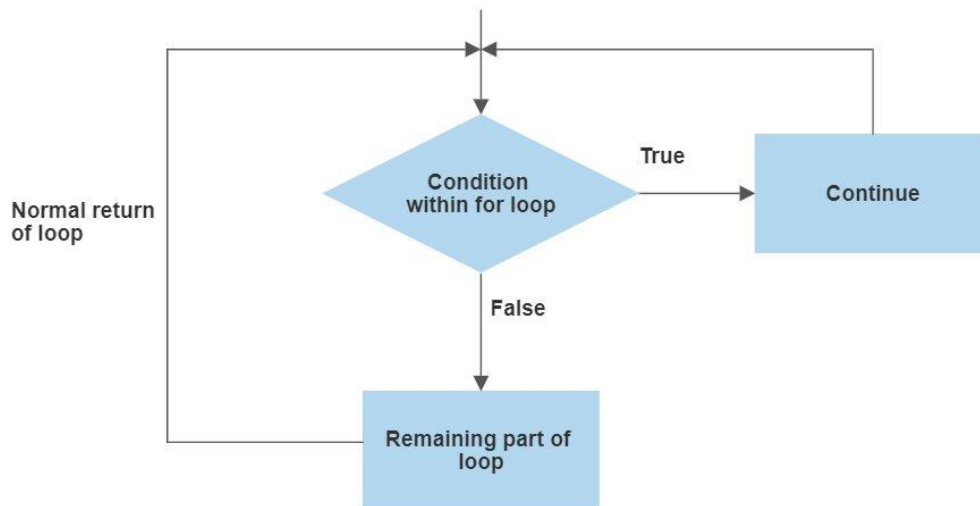
B. continue in C

Continue statement in C is used within looping constructs (**for, while & do-while loops**). When the continue statement is encountered, the current iteration of the loop is terminated immediately, and the loop's controlling expression is re-evaluated to determine if the loop should continue with the next iteration.

Syntax

`continue;`

Flowchart



For example, let's take a scenario

A girl wants to create a program that prints even numbers between 1 and 50.

```
#include <stdio.h>

int main() {
    int i;

    printf("Printing even numbers between 1 and 50:");

    for (i = 1; i <= 50; i++) {
        if (i % 2 != 0) {
            continue; // Skip odd numbers
        }
        printf("%d", i);
    }

    return 0;
}
```

Output

Printing even numbers between 1 and 50:

2
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
40
42
44
46
48
50

Using the continue statement, the program effectively skips odd numbers and prints only even numbers within the specified range.

C. goto statement in C

The goto statement in C provides a way to jump from one point in the code to another based on a label. It's a way to direct the control flow to a labelled code section.

While goto can be useful in some situations, it's generally considered a controversial construct in higher-level programming. Overusing or misusing goto can lead to "spaghetti code," which is hard to read, maintain, and debug.

Syntax

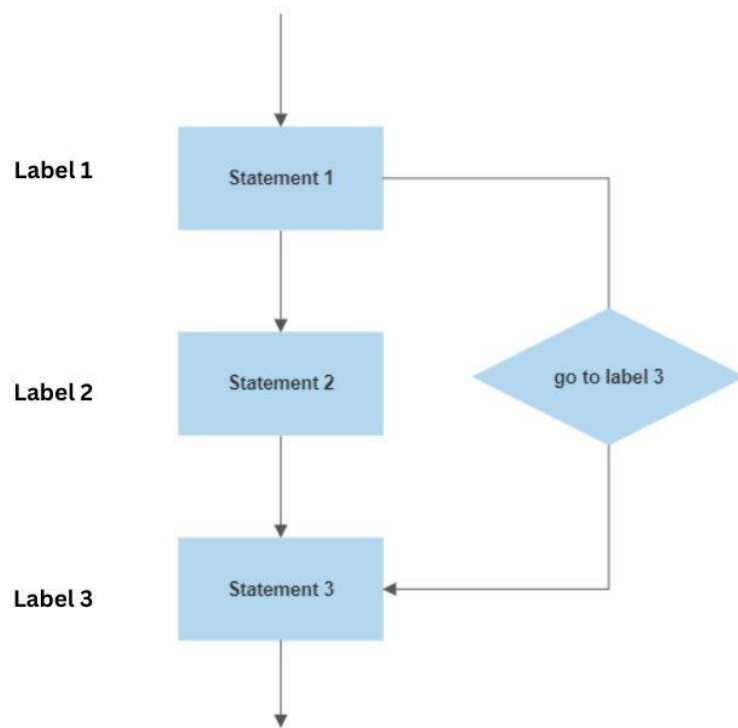
goto label_name;

...

label_name:

// code to be executed after the goto

Flowchart



For example

```
#include <stdio.h>

int main() {
    int num = 0;

    print_num:
    if (num < 7) {
        printf("%d ", num);
        num++;
        goto print_num; // Jump back to the label and execute the code again
    }

    return 0;
}
```

Output

0 1 2 3 4 5 6

The program uses a goto statement to create a loop-like structure when we run the code. The loop prints the value of num as long as it is less than 7 and then increments num by 1 in each iteration.

D. return statement in C

We use the return statement in C is used to terminate the execution of a function and return a value to the caller. We mainly use it to provide a result back to the calling code.

Syntax

*For a function that doesn't return a value (**void type**):*

return;

For a function that returns a value:

return expression;

Let's see an example

```
#include <stdio.h>

int add(int a, int b) {
    return a + b; // Returns the sum of a and b to the caller.
}

int main() {
    int result = add(4, 8); // Calls the 'add' function and stores the returned value in 'result'.
    printf("The sum is: %d", result);
    return 0;
}
```

Output

The sum is: 12

The code calls the add function to calculate the sum of two numbers and then displays the result using printf. The output shows the sum of 4 and 8 as calculated by the add function.